Claude Code ワークフロー

コーディングエージェント入門

伊藤 恒平

2026

コーディングエージェントとは

定義

AI がターミナル（黒い画面のコマンド入力画面）上でコードを読み書きし、ビルド・テスト・変更履歴の記録まで**自律的に実行**するツール

	チャット AI	**コーディングエージェント**
動作	対話（コピペが必要）	ファイルを直接編集
範囲	1 ファイルの断片	プロジェクト全体
実行	ユーザーが手動実行	ビルド・テストも自律実行
履歴管理	関与しない	変更の記録・共有まで

なぜターミナルで動くのか: エディタに依存しないので、**どんな開発環境でも同じように使える**

Claude Code とは

概要

Anthropic 社が提供するコーディングエージェント。ターミナルから Claude を呼び出し、コードの読み書きからテスト・変更記録まで自律的に実行する

機能	説明
指示書ファイル	プロジェクトのルールを書いたファイルを毎回読む
記憶の永続化	会話で学んだことを次回以降も覚えている
計画モード	コードを書かずに調査・設計だけさせるモード
Git 連携	変更の記録・共有を自動で行う
自動化の仕組み	繰り返す作業やチェックを自動実行

※ 各機能の詳細は後半で解説　インストール: `npm install -g @anthropic-ai/claude-code`　起動: `cd your-project && claude`

なぜ「計画」が必要か

AI に自由にコードを書かせると何が起きるか

- 意図と違う設計で大量のファイルを変更 → **手戻りコスト大**
- テストなしで実装を進める → バグの発見が遅れる
- 既存コードの意図を誤解 → 動いていた機能が壊れる

解決策:「承認」ステップを挟む

1. AI に計画を出させる（コードは書かせない）
2. 人間が計画をレビューして承認/修正
3. 承認後に初めて実装を開始

原則:「考える」と「書く」を分離する。これが手戻りを防ぐ最大のコツ

Plan Mode — 設計してから書く

Plan Mode（計画モード）とは

Shift+Tab キーで切り替え。コード編集を**できない**読み取り専用モード。AI に調査・設計だけをさせ、方針が固まってから実装に移る

有効な場面

- 複数ファイルにまたがる変更
- 設計の判断が必要なとき
- 失敗コストが高い作業
- 初めて触るプロジェクト

Plan Mode でできること

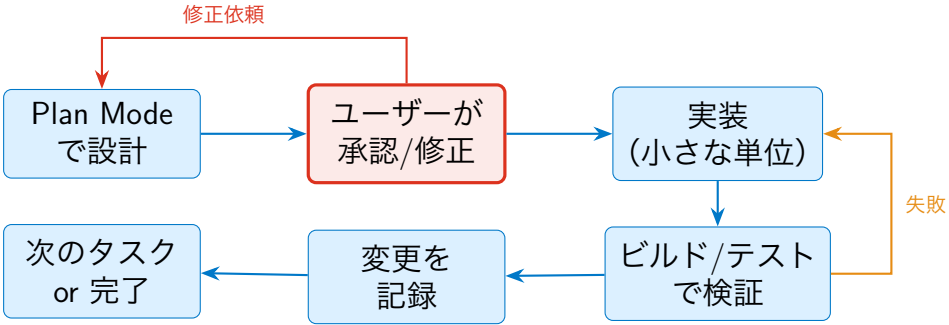
ファイルを読んで調査する
プロジェクトの構造を分析する
変更計画を作って提案する

× ファイルの編集・作成

× コマンドの実行

Tips: 大きな変更ほど計画モードが効く。5 分の計画で 1 時間の手戻りを防げる

Plan → Approve → Implement サイクル



赤枠 = 人間の承認ポイント。ここで方向修正できるから安全に AI を使える

Plan Mode の実践例

例:「CSV を読み込んでグラフ表示する機能を追加して」→ 計画モードで出る計画:

```
## Plan: CSV 読み込み + グラフ表示機能

### 変更対象ファイル
1. src/csv_reader.py   (新規) - CSV 読み込み処理
2. src/plot.py         (新規) - グラフ描画処理
3. src/main.py         (既存) - メニューに項目追加
4. tests/test_csv.py   (新規) - 読み込みのテスト

### 設計判断
- CSV ライブラリ: 標準の csv モジュールを使用
- グラフ描画: matplotlib を使用
- ファイル選択: コマンドライン引数で指定

### 影響範囲
- 既存の機能には影響なし
- requirements.txt に matplotlib を追加
```

ユーザーが「3 ファイルに分けず 1 ファイルにまとめて」→ 計画を修正 → 承認 → 実装開始

なぜ Git（変更履歴の管理）が重要か

AI がファイルを自律的に変更する世界では

変更履歴がなければ「何が変わったか」「なぜ変わったか」が追えない。

Git なしで AI にコードを書かせるのは、セーブなしでゲームをするようなもの

Git が提供する安全網

Git = 変更履歴を記録・管理するツール

- 変更の差分を確認できる
- いつでも前の状態に戻せる
- 誰が何を変えたか記録される

Claude Code の Git 連携

- 記録メッセージを自動作成
- 統一された記録形式に準拠
- 変更の要約を自動生成

原則: AI と協働するなら Git は必須。「こまめに記録」が最大の保険

コミット戦略

コミット（変更の記録）のルールを決めておく

「何をなぜ変えたか」が伝わる記録を残す。Claude Code が自動でこの形式に従う:

```
feat(csv): add CSV import and graph display
                    ← 種類(範囲): 変更の要約 (1行)
Changes:
- CSV 読み込み処理を実装      ← 何を変えたか
- matplotlib でグラフ描画機能を追加

Next steps:                   ← 次にやるべきこと
- tests/test_csv.py でテストを実行
- 大きな CSV ファイルでの動作確認
```

記録するタイミング:

- 1 機能 = 1 記録（小さな単位で）
- テスト成功後
- 作業の区切り・中断時

Next steps（次の作業）の効果:

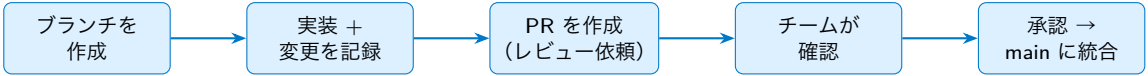
- 次回起動時の再開ポイントになる
- 具体的な検証項目を含む
- AI が「前回の続き」を即座に把握

GitHub 連携 — 変更をチームに共有する

Pull Request（PR）＝「この変更をレビューしてください」という依頼

「この変更で PR を作成して」と Claude Code に指示するだけで:

1. 変更内容を分析して説明文を自動生成
2. GitHub 上に PR（変更のレビュー依頼）を作成
3. チームメンバーのレビューコメントへの対応も可能



Tips: Claude Code はレビューコメントを読んで修正も自動で作れる

安全な運用 — main ブランチを直接触らない

main（main）を直接変更しない

Git では「main」と「ブランチ」を分けられる。AI に main を直接変更させると、レビューなしで反映されてしまう。必ずブランチ → レビュー → 統合の流れを守る

推奨フロー

1. ブランチを作成
2. その枝で実装・テスト・記録
3. レビュー依頼を作成
4. 確認・承認後に main へ統合
5. ブランチを削除

AI にやらせないことを決める

設定ファイルで制御:

- main への直接書き込みをブロック
- 使えるコマンドを制限
- GitHub 側でも main を保護できる

原則: AI に「何をさせるか」だけでなく「何をさせないか」も設定する

CLAUDE.md — AI への指示書ファイル

CLAUDE.md とは

プロジェクトのフォルダに置くテキストファイル（`CLAUDE.md`）。
Claude Code は起動するたびにこのファイルを**最初に読む**。
ここに書いたルールに AI が従うようになる

書くべき内容

- ビルド・テストの実行コマンド
- コードの書き方のルール・変更記録の形式
- **やってはいけないこと**（例:「このファイルは変更禁止」）← 最も効果が高い
- プロジェクトの構造の説明

重要: CLAUDE.md を書く ＝ **AI に知識を教える。**ドキュメント化の手間が「AI の教育投資」として返ってくる

Memory（メモリ機能）— 会話を超えて覚えている仕組み

Memory とは

Claude Code が会話の中で学んだことを**ファイルに保存**し、次回以降も自動的に読み込む仕組み。PC のメモリ（RAM）ではなく、独自の「記憶ファイル」

4 つの種類:

種類	何を覚えるか
user	あなたの専門性・好み
feedback	「こうしないで」という修正指示
project	進行中の作業・期限
reference	参考 URL やツール名

feedback の例

テストは必ず実行してから
変更を記録すること。
→ 以降ずっとこのルールに従う

ファイル名は日本語禁止。

→ 次回から自動的に従う

最も強力なのは feedback。一度「こうしないで」と伝えれば、**次回以降すべての会話**で Claude が従う

Claude Code にとっての「知識」とは

Claude Code は毎回まっさらな状態で起動する

前回の会話は覚えていない。ではどうやってルールを知るのか？ → **ファイルに書かれた情報を毎回読み込む**ことで知識を得る

知識の種類	具体例
プロジェクトのルール	「テストは pytest で実行」「変更記録は英語で書く」
過去の失敗から学んだこと	「この関数は変更禁止」
ユーザーの好み	「説明は日本語で」「コメントは簡潔に」
カスタムコマンド	`/commit` で自動記録 等

これらは**すべてファイルとして保存**されている。置き場所で「持ち運べるか」「この PC だけか」が変わる → 次ページ

持ち運べる知識 vs この PC だけの知識

持ち運べる知識（Git 管理）

プロジェクトのフォルダ内に保存。

`git clone` すれば別 PC でも復元

ファイル	役割
`CLAUDE.md`	プロジェクトのルールブック
`.claude/skills/`	カスタムコマンド
`.claude/settings.json`	権限・自動化の設定

この PC だけの知識（ローカル）

あなたの PC の中だけに保存。

別の PC には**引き継がれない**

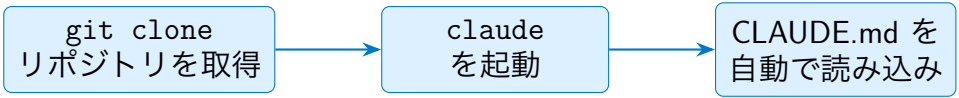
ファイル	役割
Memory	過去の会話で学んだこと
グローバル CLAUDE.md	全プロジェクト共通の個人設定

重要: Memory の大事なルールは CLAUDE.md にも書く → 持ち運べる知識になる

git clone すれば AI の知識も一緒に来る

別の PC やチームメンバーへの共有

CLAUDE.md やカスタムコマンドは Git 管理。クローンすれば **AI の知識も自動的に手に入る**

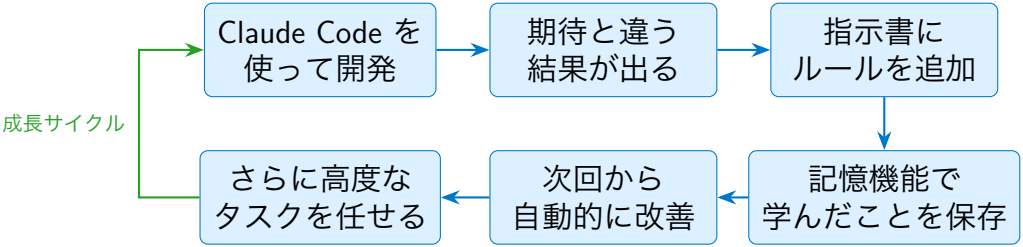


→ 初日から AI がルールを理解

GitHub = AI の知識の共有拠点

- 新メンバーがクローン → 同じルールを即座に共有
- 指示書の改善をレビュー依頼で提案 → チームで AI ルールを議論

Claude Code を育てる



1 ヶ月目: ビルドコマンドと基本規約だけの CLAUDE.md

3 ヶ月目: 失敗事例から構築されたチェックリスト・禁止事項の集大成

投資対効果: ドキュメント化の時間は「AI への教育投資」。使い込むほど Claude は賢くなる

Hooks（フック）とは —「○○したら自動で△△する」

Hooks とは

Claude Code が何か操作をする**前後に、決めておいた処理を自動実行**する仕組み。
例:「ファイルを書き換えたら自動でコード整形を走らせる」
例:「危険なコマンドを実行しようとしたらブロックする」

タイミング	何ができるか
操作の前	危険な操作を事前にブロック
操作の後	コード整形・書式チェックを自動実行
通知時	長い処理の完了をデスクトップに通知
応答完了後	自動でテストを実行

設定場所: プロジェクトの設定ファイル（`.claude/settings.json`）。一度設定すれば毎回自動で動く

Hooks の作り方と活用例

例 1: 自動フォーマット

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Write|Edit",
      "command": "npx prettier
        --write $TOOL_INPUT_PATH"
    }]
  }
}
```

例 2: 危険コマンドブロック

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "command": "if echo
        $TOOL_INPUT | grep -q
        'rm -rf'; then
        exit 2; fi"
    }]
  }
}
```

他の例: 完了通知（長い処理が終わったらデスクトップ通知）/ 自動テスト（作業完了後にテスト実行）

Skills（スキル）とは — 自分だけのコマンドを作る

Skills とは

「変更を記録して」のような複雑な手順を、`/commit` と打つだけで実行できる仕組み。

手順書ファイル（`SKILL.md`）をプロジェクト内に置いて定義する

手順書ファイルの中身

```
---
name: commit
description: ルールに従って
  変更を記録する
allowed-tools: [Bash, Read]
---
1. 変更内容を確認
2. 記録メッセージ作成
3. 変更を記録
```

最初から使えるスキル

`/commit` — 変更を自動で記録

`/simplify` — コード品質チェック

`/batch` — 大量の変更を一括処理

Tips: 繰り返す作業はスキルに

Subagent（サブエージェント）— AI の分身を使う

Subagent とは

Claude Code が**別の AI を一時的に起動して作業を任せる**仕組み。人間でいう「部下に調査を頼む」ようなもの。本体は別の作業を続けられる

種類	何をさせるか
汎用タイプ	複雑な作業を丸ごと委任
調査タイプ	コードを読んで構造を調べる（編集はしない）
設計タイプ	変更計画を考える（編集はしない）

活用例

- **複数のテストを同時実行** — 待ち時間を短縮
- **大きなプロジェクトの調査** — 結果だけを受け取る
- **裏でビルド** — 完了したら通知が届く

コンテキスト管理

コンテキスト（AI が一度に扱える情報量）には上限がある

会話が長くなると、AI は古い指示を忘れたり精度が落ちたりする。コンテキストを上手に管理することが大事

対策	説明
`/compact`（圧縮）	会話履歴を AI に要約させて短くする
サブエージェントに委任	大量の調査結果を本体に持ち込まない
Memory に保存	毎回伝え直す情報は記憶ファイルに保存
CLAUDE.md を簡潔に	毎回読むファイルなので短く保つ

Tips: `/compact` で圧縮。大事な情報は Memory に保存しておけば圧縮後も残る

初心者がやりがちな失敗

失敗パターン	何が起きるか
計画なしでいきなり実装させる	意図と違う設計で大量変更 → やり直し
変更履歴を管理しない	AI の変更を追えない → 戻せない
指示書ファイルを書かない	毎回同じ指示を繰り返す → 非効率
main に直接変更を送る	レビューなしで本番反映 → 障害リスク
自動化を使わず手作業を繰り返す	書式チェック忘れ・整形崩れが頻発
記憶機能を活用しない	同じミスを何度も指摘する必要がある

共通の原因: AI を「魔法の箱」として扱ってしまう →**対策:** 安全装置（計画・変更管理・指示書）を先に整える

Top 10 Tips

1. **計画してから書かせる** — 5 分の計画で 1 時間の手戻りを防ぐ
2. **変更は必ず Git で記録** — こまめに記録、Next steps を含める
3. **ブランチ → レビュー → main** — main を直接触らない
4. **指示書（CLAUDE.md）は AI への教育投資** — 書くほど賢くなる
5. **「やるな」は「やれ」より効果的** — 禁止事項を明記する
6. **記憶機能で同じミスを防ぐ** — 一度伝えれば以降ずっと従う
7. **Claude Code を育てる** — 失敗するたびにルールを追加
8. **フック（Hooks）で自動化** — 手作業の繰り返しを排除
9. **スキル（Skills）で 1 コマンドに** — 複雑な手順をまとめる
10. **GitHub = AI の知識の共有拠点** — クローンすれば知識も一緒に来る

参考資料 / はじめよう

リソース	URL
公式ドキュメント	https://docs.anthropic.com/en/docs/claude-code
ベストプラクティス	https://docs.anthropic.com/en/docs/claude-code/best-practices
Claude Code GitHub	https://github.com/anthropics/claude-code

Claude Code を始めるには

```
npm install -g @anthropic-ai/claude-code
cd your-project && claude
```